

Methods & Method Types

What is a Method?

In real life, we divide work into steps.

Example:

- Cooking food
- Washing clothes
- Paying bills

Each step has a purpose.

In Java, a **method is a block of code that performs a specific task.**

Instead of writing the same code again and again, we put it inside a method and reuse it.

(If this feels new, don't worry. We will go step by step.)

Why Methods are Needed

Methods help us:

- Reduce repeated code
- Organize logic properly
- Make programs easy to read and fix

Without methods:

- Code becomes long
- Same logic is written many times
- Errors are harder to find

That's why methods are a core part of Java.

Simple Method Example

```
public static void greet() {  
    System.out.println("Hello User");  
}
```

Let us understand slowly:

- public - method can be accessed
- static - can be called without object
- void - returns nothing
- greet - method name

- `{ }` - method body

This method just prints a message.

Calling a Method

Creating a method is not enough.

We must **call the method** to run it.

Example:

```
greet();
```

When this line runs:

- Control goes to the method
- Code inside method executes
- Control comes back

(Beginners often forget to call methods. Always remember this step.)

Method with Parameters

Sometimes, methods need input.

Example:

- Add two numbers
- Print a name

This input is called **parameters**.

Example:

```
public static void printName(String name) {  
    System.out.println(name);  
}
```

Explanation:

- String - data type
- name - parameter
- Value is passed during method call

Method call:

```
printName("Java");
```

Method with Return Value

Some methods return a result.

Example:

- Sum of numbers
- Calculated value

Example:

```
public static int add(int a, int b) {  
    return a + b;  
}
```

Explanation:

- int - return type
- return sends value back to caller

Method call:

```
int result = add(10, 20);
```

(If return feels confusing now, just remember it sends data back.)

void vs return Methods

void Method	return Method
No value returned	Value returned
Uses void	Uses data type
Output only	Output + value

Both are used based on need.

Static Methods

A **static method** belongs to the class.

It can be called:

- Without creating object
- Directly using method name

Example:

```
public static void show() {  
    System.out.println("Static method");  
}
```

Called as:

```
show();
```

(Main method itself is static. That's why it runs first.)

Non-Static Methods (Basic Idea)

A **non-static method** belongs to an object.

Example:

```
void display() {  
    System.out.println("Non-static method");  
}
```

To call it:

- Object is required

(Don't worry, this will be clearer in OOP repository.)

Method Overloading (Basic Awareness)

When:

- Method name is same
- Parameters are different

This is called **method overloading**.

Example:

```
add(int a, int b)  
add(int a, int b, int c)
```

Java decides which method to call based on parameters.

(This is just awareness. Detailed part comes later.)

Remember This

Methods:

- Break code into small parts
- Make programs reusable
- Improve readability

If you understand **method creation** + **method call**, you are doing great.